Agents

A - Alice
B - Browser with session and local storage, IndexedDB, WebCrypto API, and libsodium and libcrux libraries
M - WebAuthn authenticator (passkey manager)
S - Quick Crypt server

Assumptions: Protocol notations show honest parties communicating over intact TLS. The security claims under other assumptions are in the ProVerif models.

Browser and Libsodium Functions

C - Constant-time comparator from libsodium
C(value1, value2)
D_H - BLAKE2b KDF from libsodium (`crypto_kdf_derive_from_key`)
D_H(subkey number, purpose, key), 256 bit output, except n_{IVH}:
as long as n_{IV}, derived as at least 128 bits, then truncated to that length
D_P - PBKDF2-HMAC-SHA512 key derivation FIPS-180-4 from WebCrypto
D_P(material, salt, iterations)
D_S - HMAC-SHA-512 from WebCrypto
D_S(data, key)
E_a - Symmetric cipher using AEAD algorithm a
E_a(message, IV, additional data, key)
a is one of:
1. AES-256 in Galois/Counter mode from WebCrypto
2. XChaCha20 with Poly1305 MAC from libsodium
3. AEGIS 256 from libsodium

G - HMAC-SHA-512 key generator from WebCrypto
G()
H_{key} - BLAKE2b keyed hash (MAC) generator from libsodium
H_{key}(data ...), each data argument hashed in order
key $\emptyset$ for unkeyed
H_2 - SHA-256 from WebCrypto
H_2(data)
PAD - Appends 0 to 15 `0xFF` bytes to reach a multiple of 16, max 223
PAD(data)
P_K - ML-DSA-65 (FIPS 204) keypair generator from libcrux
P_K(seed)
P_S - ML-DSA-65 (FIPS 204) randomized signature generator from libcrux
P_S(secret key, message, context)
P_V - ML-DSA-65 (FIPS 204) signature verifier from libcrux
P_V(public key, message, signature, context)
R - Cryptographic pseudo-random generator: libsodium in B, AWS KMS or WebCrypto in S
R(bits)
V - MAC validator: recomputes the H tag of data under key and compares it to tag using C
V(data, key, tag)

Notes: D_H must be collision resistant for its key argument, not merely pseudorandom, or two cipher keys could share one k_C. $x == y$ compares x and y, and $=$ alone assigns.

Cipher Variables

a - Symmetric AEAD cipher and mode: [1, 2, 3]
ad_F - Additional data included in ciphertext
cd - Cipher data
cd_0 - Cipher data block 0
cd_N - Cipher data block N
cd_u - u_c encrypted under session key k_L. See u_c Encryption at Rest for $k_L, k_{KS}, pkId, s$, and Persisted State for $B.sess, B.key$
e_l - Extra key material length: 0 (fixed)
err - Error message and exit
f - Block flags: 1 (TERM) on the final block of a loop, else 0
h - Password hint text
h_E - Encrypted hint
h_El - Encrypted hint length
i - PBKDF2-HMAC-SHA512 iteration count: min 420,000 max 4,294,000,000
k_H - 256 bit ephemeral hint cipher key
k_{M0} - 256 bit ephemeral message block0 cipher key
k_{MN} - 256 bit ephemeral message blockN cipher key
k_S - 256 bit ephemeral MAC key
k_C - 256 bit key commitment, the only D_H output stored in cd
k_Cl - k_C byte length: 32
kp_B - Key purpose text: "Blck_Key"
kp_C - Key purpose text: "Cmit_Key"
kp_H - Key purpose text: "Hint_Key"
kp_S - Key purpose text: "Sign_Key"
l - Payload length
le - Loop end (1-16), stored as $le - 1$ in the high nibble of one byte
le_L - Preceding loop's le
lp - Loop count (1-16), stored as $lp - 1$ in the low nibble of that byte
lp_L - Preceding loop's lp
m - Clear text
m_0 - Clear text block 0
m_N - Clear text block N
m_E - Block of encrypted message
n_{IV} - Pseudo random initialization vector
$n_{IV}l$ - n_{IV} bit length: [96, 192, 256]
n_{IVH} - Hint initialization vector: $n_{IV}l$ bits
n_S - 128 bit pseudo random salt
N - Block number
p - Password text
p_l - p byte length
r - 384 bits of pseudo random data
t - 256 bit MAC tag
t_L - Preceding 256 bit MAC tag, or the single byte `0x00` at the start of a loop
u_c - 256 bit user credential in memory
v - Cipher data version: 8

Notes: A bit range $x[i : j)$ includes bit i and excludes bit j. For fixed empty fields, only a length of 0 is included.

Byte layout: Field widths and byte order are given by the cipher data structure table on the protocol page and by the Hex Fiend template.

Message Encryption by A

A *is authenticated*
$B \leftarrow B.sess : u_i, pkId, cd_u$
$B \leftarrow B.key[s] : k_{KS}$
$B : k_L = D_S(s \parallel pkId, k_{KS})[256 : 512)$
$B : u_c = cd_u$ decryption under k_L
$A \rightarrow B : m, i, le$
$v = 8$
$lp = 1$
$LOOP : B$ compute
    $t_L =$ `0x00`
    $A \rightarrow B : p, h, a$
    $r = R(384)$
    $n_S = r[0 : 128)$
    $n_{IV} = r[128 : 128 + n_{IV}l)$
    $n_{IVH} = D_H(2, kp_H, H_{n_{IV}}(n_S, a, v, lp, e_l))$
    $k_H = D_H(1, kp_H, H_{u_c}(n_S, a, v, lp, e_l))$
    $k_S = D_H(1, kp_S, H_{u_c}(n_S, a, v, lp, e_l))$
    $k_{M0} = D_P(p_l \parallel p \parallel u_c \parallel a \parallel v \parallel lp \parallel e_l, n_S, i)$
    $k_C = D_H(1, kp_C, H_{k_{M0}}(n_S))$
    $m_0 \parallel ... \parallel m_N = m$
    $cd = cd_0 \parallel ... \parallel cd_N$
    $m = cd$
    $lp = lp + 1$
    goto LOOP if lp $\leq le$
$A \leftarrow B : cd$

Block 0 (cd_0) Encryption by B

$h_E = E_a(PAD(h), n_{IVH}, \emptyset, k_H)$
$h_El = len(h_E)$
$f = 1$ if $\boxed{\text{TERM}}$ else 0
$ad_F = f \parallel a \parallel n_{IV} \parallel n_S \parallel i \parallel le \parallel lp \parallel h_El \parallel h_E \parallel k_Cl \parallel k_C$
$m_E = E_a(m_0, n_{IV}, ad_F, k_{M0})$
$l = len(ad_F \parallel m_E)$
$t = H_{k_S}(v \parallel l \parallel ad_F \parallel m_E \parallel t_L)$
$t_L = t$
$cd_0 = t \parallel v \parallel l \parallel ad_F \parallel m_E$

Block N (cd_N) Encryption by B

$r = R(384)$
$n_{IV} = r[0 : n_{IV}l)$
$f = 1$ if $\boxed{\text{TERM}}$ else 0
$ad_F = f \parallel a \parallel n_{IV}$
$k_{MN} = D_H(N, kp_B, H_{k_{M0}}(n_S, a, v, lp, e_l))$
$m_E = E_a(m_N, n_{IV}, ad_F, k_{MN})$
$l = len(ad_F \parallel m_E)$
$t = H_{k_S}(v \parallel l \parallel ad_F \parallel m_E \parallel t_L)$
$t_L = t$
$cd_N = t \parallel v \parallel l \parallel ad_F \parallel m_E$

Message Decryption by A

A is authenticated
$B \leftarrow B.sess : u_i, pkId, cd_u$
$B \leftarrow B.key[s] : k_{KS}$
$B : k_L = D_S(s \parallel pkId, k_{KS})[256 : 512)$
$B : u_c = cd_u$ decryption under k_L
$A \rightarrow B : cd$
$LOOP : B$ compute
    $t_L =$ `0x00`
    $cd_0 \parallel ... \parallel cd_N = cd$
    $t, v, l, ad_F, m_E = cd_0$
    $f, a, n_{IV}, n_S, i, le, lp, h_El, h_E, k_Cl, k_C = ad_F$
    $k_S = D_H(1, kp_S, H_{u_c}(n_S, a, v, lp, e_l))$
    check $V(v \parallel l \parallel ad_F \parallel m_E \parallel t_L, k_S, t)$ else *err*
    $t_L = t$
    check $k_Cl == 32$ else *err*
    $n_{IVH} = D_H(2, kp_H, H_{n_{IV}}(n_S, a, v, lp, e_l))$
    $k_H = D_H(1, kp_H, H_{u_c}(n_S, a, v, lp, e_l))$
    $h = PAD^{-1}(E_a^{-1}(h_E, n_{IVH}, \emptyset, k_H))$
    check $lp == le$ for the outermost loop else *err*
    check $lp == lp_L - 1$ and $le == le_L$ for each inner loop else *err*
    $lp_L = lp;\ le_L = le$
    $B \rightarrow A : h$
    $B \leftarrow A : p$
    check $i \geq$ 420,000 else *err*
    $k_{M0} = D_P(p_l \parallel p \parallel u_c \parallel a \parallel v \parallel lp \parallel e_l, n_S, i)$
    check $C(k_C, D_H(1, kp_C, H_{k_{M0}}(n_S)))$ else *err*
    $m = m_0 \parallel ... \parallel m_N$
    check the last decoded block had $f == 1$ else *err*
    $cd = m$
    goto LOOP if lp > 1
$A \leftarrow B : m$

Block 0 (m_0) Decryption by B

$m_0 = E_a^{-1}(m_E, n_{IV}, ad_F, k_{M0})$

Block N (m_N) Decryption by B

check the preceding block had $f == 0$ else *err*
$t, v, l, ad_F, m_E = cd_N$
$f, a, n_{IV} = ad_F$
check $V(v \parallel l \parallel ad_F \parallel m_E \parallel t_L, k_S, t)$ else *err*
$t_L = t$
check a and v equal those of cd_0 else *err*
$k_{MN} = D_H(N, kp_B, H_{k_{M0}}(n_S, a, v, lp, e_l))$
$m_N = E_a^{-1}(m_E, n_{IV}, ad_F, k_{MN})$

u_c Encryption at Rest Variables

a - Symmetric AEAD cipher and mode: 2
ad_F - Additional data included in ciphertext
cd_p - u_c encrypted under PRF output p_R
cd_r - u_c encrypted under recovery secret $u_r \parallel u_i$
cd_u - u_c encrypted under session key k_L
cd_x - Encrypted u_c cipher data: one of cd_u, cd_p, cd_r
e - Extra key material: u_i (fixed)
e_l - e byte length: 16 (fixed)
err - Error message and exit
f - Block flags: 1 (fixed)
$h_E l$ - Encrypted hint length: 0 (fixed)
i - PBKDF2 iteration count: 0 (fixed)
k - 256 bit master key: one of k_L, p_R, or $u_r \parallel u_i$
$k_C l$ - Key commitment length: 0 (fixed)
k_{KS} - HMAC-SHA-512 non-extractable CryptoKey
k_L - 256 bit per login master key
k_M - 256 bit ephemeral message cipher key
k_S - 256 bit ephemeral MAC key
kp_M - Key purpose text: "Cphr_Key"
kp_S - Key purpose text: "Sign_Key"
l - Payload length
le - Loop end: 1 (fixed)
lp - Loop count: 1 (fixed)
m_E - Encrypted message
n_{IV} - Pseudo random initialization vector
$n_{IV} l$ - n_{IV} bit length: 192
n_S - 128 bit pseudo random salt
p_R - 256 bit passkey PRF output
$pkId$ - Passkey credential identifier
r - 320 bits of pseudo random data
s - IndexedDB slot label: "user-cred-key"
t - 256 bit MAC tag
u_c - 256 bit user credential (plaintext)
u_i - User id
u_r - 128 bit recovery id
v - Cipher data version: 8

Format: At rest data uses the same wire format as messages, with the fields above fixed as listed. For cd_r this puts u_i in both k and e, which keeps the derivation identical across all three cd_x.

u_c Encryption Under k by B

where $(k, cd_x) \in \{(k_L, cd_u), (p_R, cd_p), (u_r \parallel u_i, cd_r)\}$
$r = R(320)$
$n_S = r[0 : 128)$
$n_{IV} = r[128 : 128 + n_{IV} l)$
$k_M = D_H(0, kp_M, H_k(n_S, a, v, lp, e_l, e))$
$k_S = D_H(1, kp_S, H_k(n_S, a, v, lp, e_l, e))$
$ad_F = f \parallel a \parallel n_{IV} \parallel n_S \parallel i \parallel le \parallel lp \parallel h_E l \parallel k_C l$
$m_E = E_a(u_c, n_{IV}, ad_F, k_M)$
$l = len(ad_F \parallel m_E)$
$t = H_{k_S}(v \parallel l \parallel ad_F \parallel m_E \parallel \texttt{0x00})$
$cd_x = t \parallel v \parallel l \parallel ad_F \parallel m_E$

cd_x Decryption Under k by B

where $(k, cd_x) \in \{(k_L, cd_u), (p_R, cd_p), (u_r \parallel u_i, cd_r)\}$
$t, v, l, ad_F, m_E = cd_x$
$f, a, n_{IV}, n_S, i, le, lp, h_E l, k_C l = ad_F$
$k_S = D_H(1, kp_S, H_k(n_S, a, v, lp, e_l, e))$
check $V(v \parallel l \parallel ad_F \parallel m_E \parallel \texttt{0x00}, k_S, t)$ else err
$k_M = D_H(0, kp_M, H_k(n_S, a, v, lp, e_l, e))$
$u_c = E_a^{-1}(m_E, n_{IV}, ad_F, k_M)$

Authentication Variables

ao - WebAuthn authentication options, including ch
ar - WebAuthn authentication response, including $pkId, u_i$
b - Hash binding challenges to account state
bd - HTTP request body
c_A - Sign-in count
cd_p - u_c encrypted under PRF output p_R
cd_r - u_c encrypted under recovery secret $u_r \parallel u_i$
cd_u - u_c encrypted under session key k_L
ch - 256 bit challenge value
ck - Session cookie, a signed JWT including u_i
cs - CSRF token
cw - Alice's WebAuthn authenticator credentials
mth - HTTP request method
o - Quick Crypt origin "https://quickcrypt.org"
p_R - 256 bit passkey PRF output
p_S - Fixed 256 bit PRF salt, as base64url `T89fTIZ37DRI-udjI_MBqc39I4yCyWJWHndLuYpD8yE`
pk_M - Passkey public key
$pkId$ - Passkey credential identifier
prf - Account mode: 1 for PRF, 0 for no-PRF
pth - HTTP request path
qs - HTTP request query string, omitted when empty
ro - WebAuthn registration options, including ch, u_i
rq - Authorized request, including mth, pth, bd, qs
rr - WebAuthn registration response, including $pkId, pk_M$
u_c - 256 bit user credential in memory
u_i - 128 bit user id guaranteed to be unique
u_n - A's chosen user name
u_r - 128 bit recovery id

Primes and stars: x' is a value read from persisted state, and x^* is a value received from another party. Either may differ from x.

Proof Variables

kp_R - Key purpose text: "RecovKey"
kp_U - Key purpose text: "UCredKey"
m_G - Recovery proof message for registration
m_R - Recovery proof message for account recovery and replacing words
m_U - userCred proof message for a request
n_P - 256 bit userCred proof nonce value
n_R - 256 bit recovery proof nonce value
pk_R - ML-DSA-65 recovery proof public key
pk_U - ML-DSA-65 userCred proof public key
sc_G - Recovery signature context for registration
"qcrypt/recovery/register/v1"
sc_P - userCred signature context for registration, add passkey and recovery
"qcrypt/usercred/register/v1"
sc_R - Recovery signature context for account recovery
"qcrypt/recovery/recover/v1"
sc_U - userCred signature context for authorized requests
"qcrypt/usercred/proof/v1"
sc_W - Recovery signature context for replacing words
"qcrypt/recovery/replace/v1"
sk_R - ML-DSA-65 recovery proof secret key
sk_U - ML-DSA-65 userCred proof secret key
ts_R - Recovery proof timestamp in milliseconds
ts_U - userCred proof timestamp in milliseconds
Δ - Maximum request timestamp skew: 120 seconds
δ - Delimiter byte: `\n`
σ_G - ML-DSA-65 signature for registration
σ_R - ML-DSA-65 signature for account recovery and replacing words
σ_U - ML-DSA-65 signature for an authorized request

Encodings: Every field of m_U, m_R and m_G is ASCII text: u_i, n_P, n_R, ch and pk_U are base64url, mth is uppercase, pth is percent encoded, ts_U and ts_R are decimal milliseconds, and $H_2(bd)$ is lowercase hex. In b, pk_R and pk_U are base64url and c_A is decimal.

Persisted State

$S.chal$ - Challenges by $[purpose, ch] : u_i, b$
purpose is one of "reg", "add", "auth", "recover", "confirm"
Each expires 300 seconds after it is stored
$S.nonce$ - Proof nonces by $[purpose, n_P \text{ or } n_R] : u_i$
purpose is "api" for n_P, "words" for n_R
Each expires 300 seconds after it is stored
$S.user$ - Accounts by $[u_i] : u_n, prf, pk_U, pk_R, cd_r, c_A$
$S.pk$ - Passkeys by $[u_i, pkId] : pk_M, cd_p$
$B.pin$ - Account pins in localStorage by $[u_i] : prf, pk_U$
$B.sess$ - Session in sessionStorage: $u_i, pkId, cd_u$
$B.key$ - Key slots in IndexedDB by $[s] : k_{KS}$

Notes: Reading an absent key gives $\emptyset$. $X \leftarrow T[x, y]$ deletes the entry at key x in one step if it holds y, or gives err if x is absent, expired or holds other values. $X \twoheadrightarrow T[x] : y$ stores y in one step, or gives err if x is present.

No-PRF fallback: $S.user$ holds u_c under two KMS keys in place of cd_r, and $S.pk$ holds no cd_p.

Session Start by B

$B \leftarrow B.pin[u_i] : prf', pk'_U$
check $prf' \in \{\emptyset, 0, prf\}$ else err
$(pk_U,) = P_K(D_H(1, kp_U, u_c))$
check $pk'_U \in \{\emptyset, pk_U\}$ else err
$B \rightarrow B.pin[u_i] : prf, pk_U$
$k_{KS} = G()$
$B \rightarrow B.key[s] : k_{KS}$
$k_L = D_S(s \parallel pkId, k_{KS})[256 : 512)$
$cd_u = u_c$ encryption under k_L
$B \rightarrow B.sess : u_i, pkId, cd_u$

Registration by A

$A \rightarrow B : u_n$
$B : u_c = R(256)$
$B : (pk_U,) = P_K(D_H(1, kp_U, u_c))$
$B \rightarrow S : u_n, o, pk_U$
$S : u_i = R(128);\ ch = R(256);\ b = H_\emptyset(pk_U)$
$S \rightarrow S.user[u_i] : u_n, pk_U$
$S \rightarrow S.chal[\text{“reg”}, ch] : u_i, b$
$B \leftarrow S : ro$
$B : u_r = R(128)$
$B : (pk_R, sk_R) = P_K(D_H(1, kp_R, u_r \parallel u_i))$
$B \rightarrow M : ro, p_S$
$A \rightarrow M : cw$
M create and store passkey
$B \leftarrow M : rr, p_R$
$B : cd_p = u_c$ encryption under p_R
$B : cd_r = u_c$ encryption under $u_r \parallel u_i$
$B : ts_R =$ current time
$B : m_G = u_i\delta \parallel ts_R\delta \parallel ch\delta \parallel pk_U$
$B : \sigma_G = P_S(sk_R, m_G, sc_G)$
$B : bd = (rr, u_i, ch, pk_R, ts_R, \sigma_G, cd_p, cd_r);\ rq = (\text{POST}, /\text{v1/reg/verify}, bd)$
$B \rightarrow_C S : rq$
$S \leftarrow S.user[u_i] : cd'_r$
S check $cd'_r == \emptyset$ else err
S check ts_R within Δ else err
$S : m_G = u_i\delta \parallel ts_R\delta \parallel ch\delta \parallel pk_U$
S check $P_V(pk_R, m_G, \sigma_G, sc_G)$ else err
$S : b = H_\emptyset(pk_U)$
$S \nleftarrow S.chal[\text{“reg”}, ch, u_i, b]$
S check WebAuthn validates rr against ch, o else err
$S : prf = 1;\ c_A = 1$
$S \rightarrow S.pk[u_i, pkId] : pk_M, cd_p$
$S \rightarrow S.user[u_i] : prf, pk_R, cd_r, c_A$
S create ck, cs
$B \leftarrow S : u_i^*, u_n, prf, pkId^*, cs, ck$
B check $prf == 1$ else err
B run Session Start for u_i^*
$A \leftarrow B : u_r \parallel u_i$ as BIP39

No-PRF fallback: If M returns no p_R, bd carries u_c in place of cd_p, cd_r. S checks that u_c derives pk_U, sets $prf = 0$, stores u_c, and also returns it. B checks $prf == 0$ in place of $prf == 1$ and runs Session Start with its own u_c.

Account mode: An entire account is either PRF or no-PRF mode. Downgrade from PRF to no-PRF is not supported, and B and S require every passkey to match the account's mode.

Authentication by A

$B \to S : o[, u_i]$
$S : ch = R(256)$
$S \to S.chal[\text{“auth”}, ch] : u_i$ or $\emptyset, \emptyset$
$B \leftarrow S : ao$
$B \to M : ao, p_S$
$A \to M : cw$
M sign ch
$B \leftarrow M : ar, p_R$
$B \to S : ar, ch$
$S \twoheadleftarrow S.chal[\text{“auth”}, ch, u_i$ or $\emptyset, \emptyset]$
$S \leftarrow S.pk[u_i, pkId] : pk_M, cd_p$
$S \leftarrow S.user[u_i] : u_n, prf$
S check WebAuthn validates ar against ch, o, pk_M else err
$S \to S.user[u_i] : c_A + 1$
S create ck, cs
$B \leftarrow S : u_i^*, u_n, prf, pkId^*, cd_p, cs, ck$
$B : u_c = cd_p$ decryption under p_R
B run Session Start for u_i^*

No-PRF fallback: S returns $prf = 0$ and u_c in place of cd_p, and B runs Session Start with that u_c.

Nonce Authorized Requests ($B \to_N S$)

$B \leftarrow B.sess : u_i, pkId, cd_u$
$B \leftarrow B.key[s] : k_{KS}$
$B : k_L = D_S(s \parallel pkId, k_{KS})[256 : 512)$
$B : u_c = cd_u$ decryption under k_L
$B : (pk_U, sk_U) = P_K(D_H(1, kp_U, u_c))$
$B : ts_U =$ current time
$B : n_P = R(256)$
$B : m_U = u_i\delta \parallel mth\delta \parallel pth\delta \parallel ts_U\delta \parallel n_P\delta \parallel H_2(bd)[\delta \parallel qs]$
$B : \sigma_U = P_S(sk_U, m_U, sc_U)$
$B \to S : rq, ck, cs, ts_U, n_P, \sigma_U$
S check ck, cs valid for u_i else err
S check ts_U within Δ else err
$S \leftarrow S.user[u_i] : pk_U$
$S : m_U = u_i\delta \parallel mth\delta \parallel pth\delta \parallel ts_U\delta \parallel n_P\delta \parallel H_2(bd)[\delta \parallel qs]$
S check $P_V(pk_U, m_U, \sigma_U, sc_U)$ else err
$S \twoheadrightarrow S.nonce[\text{“api”}, n_P] : u_i$ if $mth \neq$ GET

Notes: mth, pth, bd, qs, rq, ck and cs come from the calling stanza. S takes u_i from ck, and u_i and pk_U are provided to the calling stanza.

Challenge Authorized Requests ($B \to_C S$)

$B : (pk_U, sk_U) = P_K(D_H(1, kp_U, u_c))$
$B : ts_U =$ current time
$B : m_U = u_i\delta \parallel mth\delta \parallel pth\delta \parallel ts_U\delta \parallel ch\delta \parallel H_2(bd)[\delta \parallel qs]$
$B : \sigma_U = P_S(sk_U, m_U, sc_P)$
$B \to S : rq[, ck, cs], ts_U, \sigma_U$
S check ck, cs valid for u_i else err if rq requires a session
S check ts_U within Δ else err
$S \leftarrow S.user[u_i] : pk_U$
$S : m_U = u_i\delta \parallel mth\delta \parallel pth\delta \parallel ts_U\delta \parallel ch\delta \parallel H_2(bd)[\delta \parallel qs]$
S check $P_V(pk_U, m_U, \sigma_U, sc_P)$ else err

Notes: u_c, u_i, mth, pth, ch, bd, qs, rq, ck and cs come from the calling stanza. S takes u_i from ck if present, else bd, and u_i and pk_U are provided to the calling stanza.

Add Passkey by A

A is authenticated
$B : bd = \emptyset;\ rq = (\text{GET}, /\text{v1/passkeys/options}, bd)$
$B \rightarrow_N S : rq, o, ck, cs$
$S : ch = R(256);\ b = H_\emptyset(pk_U)$
$S \rightarrow S.chal[\text{“add”}, ch] : u_i, b$
$B \leftarrow S : ro$
$B \rightarrow M : ro, p_S$
$A \rightarrow M : cw$
M create and store passkey for u_i^*
$B \leftarrow M : rr, p_R$
$B \leftarrow B.sess : u_i, pkId, cd_u$
$B \leftarrow B.key[s] : k_{KS}$
$B : k_L = D_S(s \parallel pkId, k_{KS})[256 : 512)$
$B : u_c = cd_u$ decryption under k_L
$B : cd_p = u_c$ encryption under p_R
$B : bd = (rr, ch, cd_p);\ rq = (\text{POST}, /\text{v1/passkeys/verify}, bd)$
$B \rightarrow_C S : rq, ck, cs$
$S : b = H_\emptyset(pk_U)$
$S \twoheadleftarrow S.chal[\text{“add”}, ch, u_i, b]$
S check WebAuthn validates rr against ch, o else err
$S \rightarrow S.pk[u_i, pkId] : pk_M, cd_p$

No-PRF fallback: B omits p_S and cd_p; S stores the new passkey without encrypted u_c.

Replace Recovery Words by A

A is authenticated
B repeat Authentication of A (B policy, not checked by S)
$B \leftarrow B.sess : u_i, pkId, cd_u$
$B : u_r = R(128)$
$B : (pk_R, sk_R) = P_K(D_H(1, kp_R, u_r \parallel u_i))$
$B \leftarrow B.key[s] : k_{KS}$
$B : k_L = D_S(s \parallel pkId, k_{KS})[256 : 512)$
$B : u_c = cd_u$ decryption under k_L
$B : cd_r = u_c$ encryption under $u_r \parallel u_i$
$B : ts_R =$ current time
$B : n_R = R(256)$
$B : m_R = u_i\delta \parallel ts_R\delta \parallel n_R$
$B : \sigma_R = P_S(sk_R, m_R, sc_W)$
$B : bd = (pk_R, ts_R, n_R, \sigma_R, cd_r);\ rq = (\text{PUT}, /\text{v1/recover3/key}, bd)$
$B \rightarrow_N S : rq, ck, cs$
S check ts_R within Δ else err
$S : m_R = u_i\delta \parallel ts_R\delta \parallel n_R$
S check $P_V(pk_R, m_R, \sigma_R, sc_W)$ else err
$S \twoheadrightarrow S.nonce[\text{“words”}, n_R] : u_i$
$S \rightarrow S.user[u_i] : pk_R, cd_r$
$A \leftarrow B : u_r \parallel u_i$ as BIP39

No-PRF fallback: B does not decrypt cd_u or send cd_r; S stores pk_R without cd_r.

Recovery from Lost Passkey by A

$A \to B : u_r \parallel u_i$ as BIP39
$B : (pk_R, sk_R) = P_K(D_H(1, kp_R, u_r \parallel u_i))$
$B : ts_R =$ current time
$B : n_R = R(256)$
$B : m_R = u_i\delta \parallel ts_R\delta \parallel n_R$
$B : \sigma_R = P_S(sk_R, m_R, sc_R)$
$B \to S : u_i, ts_R, n_R, \sigma_R$
$S \leftarrow S.user[u_i] : pk_R$
S check ts_R within Δ else err
$S : m_R = u_i\delta \parallel ts_R\delta \parallel n_R$
S check $P_V(pk_R, m_R, \sigma_R, sc_R)$ else err
$S \twoheadrightarrow S.nonce[\text{“words”}, n_R] : u_i$
$S \to S.user[u_i] : c_A + 1$
$S \leftarrow S.user[u_i] : pk_U, c_A$
$S : ch = R(256);\ b = H_\emptyset(pk_R \parallel \text{“:”} \parallel c_A \parallel \text{“:”} \parallel pk_U)$
$S \to S.chal[\text{“confirm”}, ch] : u_i, b$
$B \leftarrow S : prf, ch, cd_r$
$B : u_c = cd_r$ decryption under $u_r \parallel u_i$
$B \leftarrow B.pin[u_i] : prf', pk'_U$
B check $prf' \in \{\emptyset, 0, prf\}$ else err
$B : (pk_U,) = P_K(D_H(1, kp_U, u_c))$
B check $pk'_U \in \{\emptyset, pk_U\}$ else err
$B \to B.pin[u_i] : prf, pk_U$
$B : bd = (u_i, ch);\ rq = (\text{POST}, \text{/v1/recover/confirm}, bd)$
$B \to_C S : rq, o$
$S \leftarrow S.user[u_i] : pk_R, c_A$
$S : b = H_\emptyset(pk_R \parallel \text{“:”} \parallel c_A \parallel \text{“:”} \parallel pk_U)$
$S \twoheadleftarrow S.chal[\text{“confirm”}, ch, u_i, b]$
S delete $S.pk[u_i, pkId]$ for every $pkId$
$S \to S.user[u_i] : c_A + 1$
$S \leftarrow S.user[u_i] : c_A$
$S : ch = R(256);\ b = H_\emptyset(pk_R \parallel \text{“:”} \parallel c_A \parallel \text{“:”} \parallel pk_U)$
$S \to S.chal[\text{“recover”}, ch] : u_i, b$
$B \leftarrow S : ro$
$B \to M : ro, p_S$
$A \to M : cw$
M create and store passkey for u_i^*
$B \leftarrow M : rr, p_R$
$B : cd_p = u_c$ encryption under p_R
$B : bd = (rr, u_i, ch, cd_p);\ rq = (\text{POST}, \text{/v1/recover/verify}, bd)$
$B \to_C S : rq$
$S \leftarrow S.user[u_i] : pk_R, c_A$
$S : b = H_\emptyset(pk_R \parallel \text{“:”} \parallel c_A \parallel \text{“:”} \parallel pk_U)$
$S \twoheadleftarrow S.chal[\text{“recover”}, ch, u_i, b]$
S check WebAuthn validates rr against ch, o else err
$S \to S.pk[u_i, pkId] : pk_M, cd_p$
$S \to S.user[u_i] : c_A + 1$
S create ck, cs
$B \leftarrow S : u_i^*, u_n, prf^*, pkId^*, cs, ck$
$B \leftarrow B.pin[u_i] : prf'$
B check $prf' \in \{\emptyset, 0, prf^*\}$ else err
B run Session Start for u_i^*, prf^*

No-PRF fallback: S returns $prf = 0$ and u_c in place of cd_r; B omits p_S and the cd_r decryption. After the new passkey, bd is (rr, u_i, ch) and S stores it without encrypted u_c.